CMDBUF RD 接收资格与容量时序分析

src aready entry 占用量 版本 1.0

cmdbuf_rd 通过一组共享 AR 接收三个 XAU 的命令。src 指明当前命令来自谁，aready 的三个位分别告诉上游哪些来源本拍可以被接收。它们共同决定是否分配 entry；一条命令只有完成 R 响应，才会释放这项容量。

判断能否接收，必须同时知道三个来源各自占了多少项，以及本拍由哪个来源回收一项。只看总数，或只检查当前来源是否少于七项，都会遗漏容量保护条件。本文按当前 RTL 展开所有容量判定类别，再用逐拍示例说明边界变化。

先明确有效 entry 的含义

本文用 N0、N1、N2 表示各 XAU 已接收但尚未通过 R 回收的命令数，总量 N=N0+N1+N2。计数从复位清空后重新开始。用户所说“AR 发出的命令数减 R 回收的命令数”，其中 AR 必须按成功握手计数，不能只数 arvalid 拉高的次数。

```
A = arvalid && (src < 3) && aready[src];
R = |(rvalid & rready);
N_next = N + A - R;
```

待发命令、已经发射但未返回的命令、数据已经存入 OTB 但未交付的命令，都占用 entry。成功发射和 MEM 数据返回本身不减少 N。pending_cnt 统计尚未成功发射的命令，与 N 是两个不同的量。

拍初占用 N0/N1/N2	总量	本拍没有 R 回收时 aready	说明
7 / 0 / 0	7	110	XAU0 达到上限；另两个来源可用首项。
3 / 3 / 1	7	111	仍有可给任一来源使用的共享余量。
4 / 4 / 0	8	100	剩余一项保留给 XAU2。
3 / 3 / 2	8	111	三个来源都已有首项，最后一项可共享。

所有向量均按 [2:0] 书写：110 表示 XAU2 和 XAU1 可接收，XAU0 不可接收。文中的 R 事件均指实际响应握手，而不是仅有 rready、MEM 返回或队头完成。

1 同一拍内的计算顺序

把 Ck 定义为第 k 个上升沿的采样点。Nk 是这个沿更新前的寄存占用；本拍 R 和 AR 在该沿被确认，沿后写成 Nk+1。表格中的“拍初”和“沿后”使用同一含义。

计算步骤	使用的信息	形成的结果
确定 R 回收	已有命令的完成状态、本源队头、rready 和 RR 起点	得到本拍 R 是否发生以及回收来源 r。
扣除本拍回收	三个拍初占用 Ni 和实际 R	Mi = Ni - 该源本拍回收数。Mi 仅用于组合准入计算。
计算 aready	Mi、总容量和每源首项保留要求	得到三个来源各自的接收资格。此处不使用当前 src 或 arvalid。
判断 AR 接收	arvalid、src 以及 aready[src]	最多接收一条共享命令。
更新占用	Mi 和实际 AR 接收来源 s	沿后 Ni_next = Mi + 该源本拍接收数。

当前 RTL 的容量公式

三个来源各保留首项资格，其余六项可共享。这里的“保留”是准入规则，不是把物理 entry 固定划分给某个来源。

```
M[i] = N[i] - (R && r_src == i);
U = M[0] + M[1] + M[2];
S = max(M[0]-1, 0) + max(M[1]-1, 0) + max(M[2]-1, 0);
aready[i] = rst_n && (U < 9) && ((M[i] == 0) || (S < 6));
N_next[i] = M[i] + (A && src == i);
```

以上是便于阅读的伪代码。实际代码位于 cmdbuf_rd_cmd.sv 的 p_4 组合逻辑以及 occ_q 的时序更新处。

本拍 AR 不提前扣掉 ready

aready 必须先表示“当前这条命令能否被接受”，随后才能判断 A。不能先把尚未确认的 AR 算进占用再决定同一次握手。接受 AR 后，寄存占用在沿后增加，影响下一拍的 ready；R 回收则已通过组合路径影响本拍 ready。

因此 aready 不是寄存输出，也不需要比 src 提前一拍拉高。上游需保证采样沿附近的 src、arvalid 和命令字段满足时序要求。本模块存在 rready 经 R 选择到 aready 的组合路径，集成时应避免上下游又把这条路径接成组合环。

2 没有 R 回收时的全部容量类别

这一页先固定 R=0，因而 M=拍初 N。合法占用要求三个来源的 max(Ni,1) 之和不超过 9。这既保证总量不超过 9，也为尚未使用容量的来源保留第一项。

总量 N	来源占用分布	arready	接收条件
0 至 6	任意合法分布	111	三个来源都能接收。
7	仅一源占七项，如 7/0/0	110 的来源置换	仅另外两个空来源能接收。
7	至少两个来源非空	111	任何来源可接收。
8	两个来源非空，如 4/4/0	100 的来源置换	仅当前空来源能接收。
8	三个来源非空	111	任何来源可接收最后一个物理空项。
9	三个来源均非空	000	没有 AR 握手；按输入约束 arvalid=0。

八种 ready 向量都可能出现

arready[2:0]	占用示例	允许来源
000	3 / 3 / 3	无
001	0 / 4 / 4	XAU0
010	4 / 0 / 4	XAU1
011	0 / 0 / 7	XAU0、XAU1
100	4 / 4 / 0	XAU2
101	0 / 7 / 0	XAU0、XAU2
110	7 / 0 / 0	XAU1、XAU2
111	3 / 2 / 1	三个来源

arready 不是独热向量。多个 ready 同时为 1，只是多个来源都有资格；AR 仍只有一组负载，每拍最多接收一条。单源最多七项也不是完整规则，例如 5/4/0 虽然每源都不超过七项，却占掉了 XAU2 的首项容量，是非法状态。

3 同拍 R 回收怎样改变接收资格

先计算拍初共享占用 S0=sum(max(Ni-1,0))。S0≤5 时，不论本拍是否回收，三个来源都可以接收。需要特别分析的是 S0=6，也就是六项共享余量全部用完。

拍初共享量	本拍回收情况	本拍 aready 的完整规则
S0≤5	无回收，或任一合法回收	111。
S0=6	没有 R	只对拍初占用为 0 的来源置 1；没有空来源时为 000。
S0=6	回收源原来至少有两项	释放了一项共享余量，三个来源均 ready，即 111。
S0=6	回收源原来只有一项	共享量仍为六；原来的空来源与刚变空的回收源 ready，其余为 0。

边界实例

拍初占用	本拍 R 来源	扣 R 后的 M	本拍 ready
7/1/1	XAU0	6/1/1	111
7/1/1	XAU1	7/0/1	010
4/4/1	XAU2	4/4/0	100
4/4/0	XAU0	3/4/0	111
7/1/0	XAU1	7/0/0	110
7/0/0	XAU0	6/0/0	111

回收“首项”与回收“额外项”有不同效果：首项释放后，该来源仍有再次使用首项的资格，不能把这个位置无条件交给别的已非空来源。因此，物理上出现空 entry 并不等于每个来源都可接收。

正常运行且占用合法时，R 一旦发生，至少一个来源 ready。aready=000 当且仅当拍初总量为 9 且本拍没有 R。满容量时，回收多项来源得到 111；回收单项来源则只得到该来源的一位 ready。

R 来源由本源队头、完成状态、rready 和 RR 共同决定。例如 4/4/1、三源队头均已完成且 ready 时，RR 选 XAU0 会使 aready=111；选 XAU2 会使 aready=100。当前 src 不参与这次 R 选择。

4 src 与进出事件的组合

arvalid	src 与 ready 的关系	AR 是否接收	占用含义
0	src 任意，包括 3	否	只可能因本拍 R 而减少。
1	src 合法且对应 ready=1	是	该来源增加一项，仍需叠加本拍 R。
1	src 合法但对应 ready=0，其他位有 1	否	允许提出请求，但这条命令未被接收。
1	arready=000	输入违约	上游已约定这种情况下 arvalid 必须为 0。
1	src=3	输入违约	当前合法事务只允许来源 0、1、2。

实际 AR A	实际 R	总量变化	各来源占用变化
0	0	不变	全部不变。
1	0	加 1	仅 src 对应来源加 1。
0	1	减 1	仅实际 R 来源减 1。
1	1，同源	不变	各源数量不变，但旧命令被新命令替换。
1	1，异源	不变	接收来源加 1，回收来源减 1；下一拍 ready 可能变化。

共享余量的变化也可以逐项追踪

AR 接收到此前为空的来源，使用它的首项，不增加共享占用；接收到此前非空的来源，共享占用加一。R 回收多项来源，共享占用减一；回收唯一一项，共享占用不变。判断 AR 的“此前”，应使用已经扣除本拍 R 的 M。

```
S_next = S0 - (R && N[r_src] >= 2)
            + (A && M[src] >= 1);
```

上游可以在未握手后切换 src，当前协议没有要求锁住失败请求直到接收。切换时必须让地址、ID、port_id、错误状态与新 src 对应。若旧请求尚未被接收，它仍由上游保存和重试。

即使 arready=111，也不是给三个来源各授予一个同拍名额。src 只选择本拍唯一负载的归属，不能用 arvalid && (|arready) 代替真正的 AR 握手条件。

5 单源占满与来源切换

W01　单源到达七项后切换来源

每列在该拍上升沿采样；沿后状态在下一列成为拍初状态

clk 采样沿	C1	C2	C3	C4	C5	C6
拍初 N0/N1/N2	6/0/0	7/0/0	7/0/0	7/1/0	7/1/0	7/1/1
R 回收来源	-	-	-	-	-	-
arvalid	1	1	1	1	1	0
src	0	0	1	1	2	-
arready[2:0]	111	110	110	100	100	000
AR 接收	1	0	1	0	1	0
沿后 N0/N1/N2	7/0/0	7/0/0	7/1/0	7/1/0	7/1/1	7/1/1
沿后总量 N	7	7	8	8	9	9

协议示意 | 三位向量的书写顺序均为 XAU2、XAU1、XAU0

图 1　从 6/0/0 开始；所有拍都没有 R 回收

C1 接收 XAU0 的第七条。C2 虽然 arvalid=1、且还有其他 ready 位，但 src=0 对应 ready=0，所以占用不变。C3 切到 XAU1 后可接收；C4 继续发 XAU1 被拒绝；C5 切到 XAU2 才能填入第九项。C6 满容量且无 R，arvalid 必须为 0。

W05　回收最后一项和回收额外项的区别

每列在该拍上升沿采样；沿后状态在下一列成为拍初状态

clk 采样沿	C1	C2	C3	C4	C5
拍初 N0/N1/N2	7/1/0	7/0/0	7/1/0	7/1/0	7/1/0
R 回收来源	1	-	0	-	-
arvalid	1	1	1	1	1
src	0	1	0	0	2
arready[2:0]	110	110	111	100	100
AR 接收	0	1	1	0	1
沿后 N0/N1/N2	7/0/0	7/1/0	7/1/0	7/1/0	7/1/1
沿后总量 N	7	8	8	8	9

协议示意 | 三位向量的书写顺序均为 XAU2、XAU1、XAU0

图 2　同样是回收一项，来源原有数量决定是否释放共享余量

C1 回收 XAU1 的唯一一项，XAU0 仍不可接收，N 从 8 降为 7。C3 回收 XAU0 的一项，先由七项降为六项，因此可同拍再接收 XAU0，新旧替换后仍为 7/1/0。

6 满容量时的两种回收

W02 满九项时回收多项来源

每列在该拍上升沿采样；沿后状态在下一列成为拍初状态

clk 采样沿	C1	C2	C3	C4	C5	C6
拍初 N0/N1/N2	7/1/1	7/1/1	6/1/2	6/2/1	5/2/1	6/2/1
R 回收来源	-	0	2	0	-	-
arvalid	0	1	1	0	1	0
src	-	2	1	-	0	-
arready[2:0]	000	111	111	111	111	000
AR 接收	0	1	1	0	1	0
沿后 N0/N1/N2	7/1/1	6/1/2	6/2/1	5/2/1	6/2/1	6/2/1
沿后总量 N	9	9	9	8	9	9

协议示意 | 三位向量的书写顺序均为 XAU2、XAU1、XAU0

图 3 回收多项来源后，任意一个来源可以同拍补入

C2 回收 XAU0，使其从七项变六项，本拍 arready=111；上游选择 XAU2 补入，总量保持九项。C3 回收 XAU2 的旧队头后，同样可以给 XAU1。假设图中被选择回收的旧队头均已完成，rready 使其具备响应资格。

W03 满九项时回收唯一一项

每列在该拍上升沿采样；沿后状态在下一列成为拍初状态

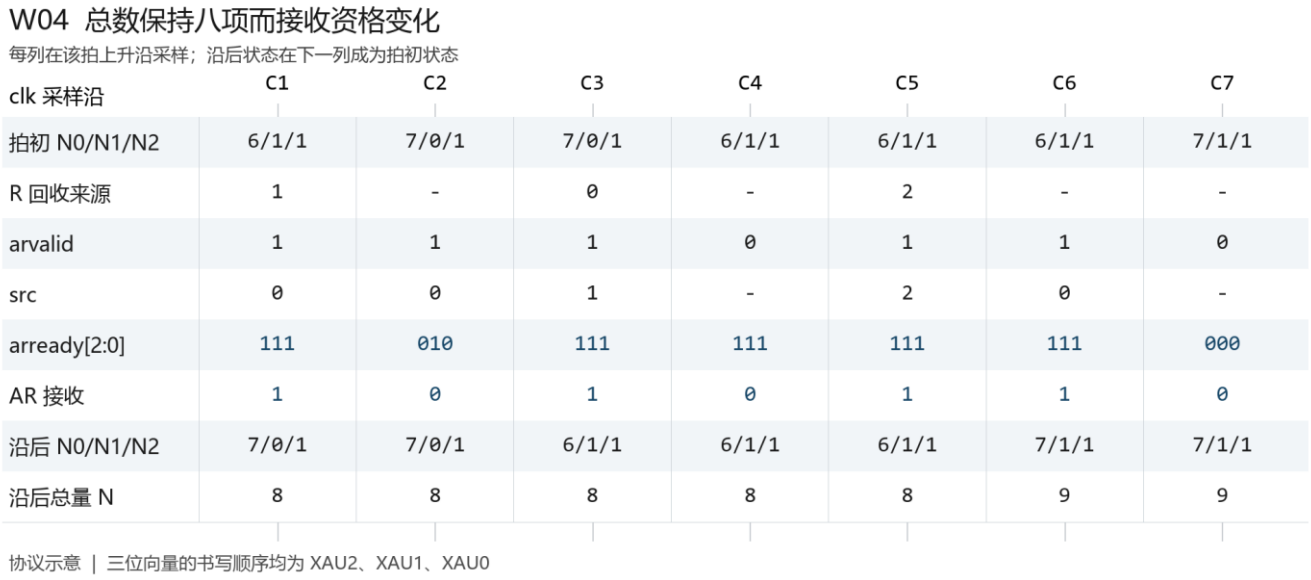
clk 采样沿	C1	C2	C3	C4	C5
拍初 N0/N1/N2	4/4/1	4/4/1	4/4/0	4/4/0	4/4/1
R 回收来源	-	2	-	-	-
arvalid	0	1	1	1	0
src	-	0	1	2	-
arready[2:0]	000	100	100	100	000
AR 接收	0	0	0	1	0
沿后 N0/N1/N2	4/4/1	4/4/0	4/4/0	4/4/1	4/4/1
沿后总量 N	9	8	8	9	9

协议示意 | 三位向量的书写顺序均为 XAU2、XAU1、XAU0

图 4 回收来源的唯一一项后，容量仍保留给该来源

C2 回收 XAU2 唯一一项后，arready 只有 bit2 为 1。src=0 的请求不能补入，因而 N 降到 8。C3 换成 XAU1 仍不能接收；C4 换成 XAU2 才恢复总量九项。这个例子中 XAU0、XAU1 都未达到七项，仍必须被阻止继续扩占。

7 总量相同不代表下一拍资格相同



clk 采样沿	C1	C2	C3	C4	C5	C6	C7
拍初 N0/N1/N2	6/1/1	7/0/1	7/0/1	6/1/1	6/1/1	6/1/1	7/1/1
R 回收来源	1	-	0	-	2	-	-
arvalid	1	1	1	0	1	1	0
src	0	0	1	-	2	0	-
arready[2:0]	111	010	111	111	111	111	000
AR 接收	1	0	1	0	1	1	0
沿后 N0/N1/N2	7/0/1	7/0/1	6/1/1	6/1/1	6/1/1	7/1/1	7/1/1
沿后总量 N	8	8	8	8	8	9	9

图 5 同进同出改变来源分布，进而改变 ready

C1 拍初是 6/1/1，总量八项。回收 XAU1 后为 6/0/1，共享占用仍为五，本拍三个来源都 ready。上游选择 XAU0 接收，沿后变成 7/0/1，总量仍为八，但共享占用升到六。

因此 C2 在没有 R 的情况下，arready 变成 010，只允许 XAU1。C2 的 XAU0 请求不能被接收。C3 回收 XAU0 的一项并接收 XAU1，分布又变回 6/1/1，下一拍三个来源都可以接收。

比较点	占用分布	总量	无额外 R 时的 ready
C1 拍初	6 / 1 / 1	8	111
C1 沿后即 C2 拍初	7 / 0 / 1	8	010
C3 沿后即 C4 拍初	6 / 1 / 1	8	111

C5 展示同源同进同出：XAU2 回收一条又接收一条，各源数量都不变，但命令、年龄及 OTB 内容已经更换。数量不变不能用于判断“本拍没有事务活动”。

判断下一拍 ready 还需要下一拍 R

表内的“下一拍 ready”只有在明确下一拍不回收时，才能由沿后占用直接给出。若下一拍又发生 R，必须重新扣除那一拍的回收后计算。不能把沿后数量机械换成一个固定、必然持续整拍的 ready。

8 MEM 返回与 entry 回收的拍间关系

W06 同源替换后等待新数据完成

每列在该拍上升沿采样；沿后状态在下一列成为拍初状态

clk 采样沿	C1	C2	C3	C4	C5	C6	C7	C8
拍初 N0/N1/N2	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1
req_success	0	0	1	0	0	0	0	0
MEM_capture	0	0	0	0	0	0	1	0
R 回收来源	2	-	-	-	-	-	-	2
arvalid	1	0	0	0	0	0	0	1
src	2	-	-	-	-	-	-	2
arready[2:0]	100	000	000	000	000	000	000	100
AR 接收	1	0	0	0	0	0	0	1
沿后 N0/N1/N2	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1	4/4/1
沿后总量 N	9	9	9	9	9	9	9	9

协议示意 | 三位向量的书写顺序均为 XAU2、XAU1、XAU0

图 6 满容量下，同源旧命令回收和新命令接收发生在 C1 与 C8

图中拍初为 4/4/1，只允许 XAU2 接收 R 响应。C1 返回 XAU2 旧命令，同时接收一条新的 XAU2 命令；唯一空出的物理 entry 在本拍被复用。各源数量仍为 4/4/1，下一拍再次没有空容量。

这条新命令最早在 C2 预选、C3 成功发射，MEM 在 C7 返回数据。C7 只能捕获，不能直接通过 R 交付，所以 arready 仍为 000，不能因 MEM 返回而提前允许新 AR。最早到 C8，已保存的数据才能由 R 返回，本拍才重新得到 arready=100。

事件	N 是否直接变化	对 ready 的影响
预选、下游仲裁失败	否	不释放容量。
下游成功发射	否	pending_cnt 减少，N 保持。
MEM 数据和 pc_err 返回	否	先存入 OTB；不能直接当成本拍 R。
数据已完成但不是本源队头	否	继续等待，不能回收该项。
本源队头完成但 rready=0	否	该源不参加本拍 R 选择。
实际 R 响应成功	是，减一	组合扣除回收后，重新计算本拍 ready。

Bank 忙、五项窗口堵住或仲裁持续失败，不会直接拉低 arready；只要容量允许，AR 仍可继续接收。反过来，pending_cnt=0 也不代表有空项：九条命令都可能已发出，却因 R 未回收而继续占满。

9 空闲 复位与长期等待

空状态与复位不是同一件事

正常运行且 N=0 时，arready=111，R 不可能合法发生；即使同拍接收新命令，也不能同拍返回这条新命令。rst_n=0 时，当前 RTL 把 arready 拉为 000 并清空占用；释放复位且状态为空后，可组合得到 111。累计 AR−R 的记账应从此次复位重新开始。

src 的无效值和未握手值

arvalid=0 时，src 不参与分配，取 0、1、2 或 3 对占用无影响。arvalid=1 时 src 必须合法。src 合法但其 ready 位为 0 的请求，不写入命令、不分配 OTB，也不改变年龄次序；即使随后换源成功接收，之前那条未握手请求也不能被误记为已经入队。

错误事务也占用容量

arresp=11 的越界命令仍按正常命令接收并送到 MEM，所以成功 AR 仍使对应来源占用加一；之后仍只由实际 R 减一。pc_err 只影响响应状态，不改变容量计数或回收时点。相同 ID 或 port_id 的多条命令也分别占用 entry，不能合并计数。

容量保护不等于系统保证等待上限

某个来源长期 rready=0 时，它的命令不能回收，但仍受最多七项及其他来源首项保留规则限制。未使用容量的其他来源仍可提交首条命令；这些来源已有一条未回收后，可能还要等共享余量才能继续提交。保护目标不是永久给每个来源留一条额外空项。

上游允许切换 src，但尚未承诺公平调度。若上游一直选择 ready=0 的来源，其他 ready 位即使为 1，也不会自动接收其他来源的请求。下游若长期不成功，或某源队头长期不能返回，同样不能据容量规则推出有限等待时间。

连续同进同出的条件

满容量连续吞吐时，每拍都必须有实际 R，并且本拍 src 处于当拍 ready 集合内。若回收多项来源，三源中任选其一可接收；若回收单项来源，只能由这个刚变空的来源补入。只满足“每拍返回一条”和“每拍提出一条请求”，仍可能因为来源不匹配而出现接收空拍。

10 场景完整性与核对入口

按三个来源的占用数穷举，合法状态共 190 种。对每种状态枚举“无 R”以及各非空来源作为 R 来源，共 631 个状态与回收组合；再枚举无效 AR 和三个合法 src 的有效 AR，排除 arready=000 时的有效 AR，共得到 2440 个动作组合。arvalid=0 的所有 src 被合并为等价类。

每个组合都核对了当前 p_4 公式与“试加一条后仍满足首项预留不变量”的独立判定一致，并检查总量、分源增减及下一状态合法性。六张时序示意由同一数据生成。这里是容量抽象的穷举分析，没有新增 RTL 仿真或形式证明，也没有把任意非空来源自动视为已具备 R 完成条件。

核对类别	必须区分的情况
请求是否接收	arvalid=0；有效且所选 ready=1；有效但所选 ready=0；全 ready=0；非法 src。
容量分布	总量 0 至 6；七项单源或多源；八项双源或三源；九项满容量。
R 释放类型	无 R；回收来源只有一项；回收来源有多项。
同拍进出	无进出；只进；只出；同源同进出；异源同进出；请求被阻但仍有 R。
共享余量边界	共享量五到六；六到五；首项回收不减少共享量；首项接收不增加共享量。
来源切换	停留在受阻来源；切到可接收来源；同总量下分布变化导致 ready 改变。
完成与回收	未发射；在途；MEM 捕获当拍；完成但非队头；队头完成但未 ready；实际 R。
容量与吞吐	七项单源保留其他两源；4/4/0；满池多项回收；满池单项回收；连续替换。

与当前源码的对应关系

文件或数据	核对内容
rtl/cmdbuf_rd_cmd.sv	p_4：先扣本拍 R，再计算三位 ready；occ_q：按实际 AR/R 更新。
formal/protocol_20261021/cmdbuf_rd_port_sva.sv	A_READY 检查完整 ready 规则；E03_SRC 与 E05_ZERO_READY 描述输入约束。
docs/timing/admission_20261021/	全部 2440 个容量动作组合 CSV、六组时序数据、分析核对记录与图示。

本分析依据 2026 年 10 月 21 日当前四态 wire 接口版本。源文件校验值记录在 analysis_audit.json；本轮未修改 RTL 或 SVA。